

Appendix C — Contract Resolutions

Oct 3, 2026 · @michael barrett

Status: Binding for P0 implementation. Companion to Spec V2 Rev 4, Phase 1 Contract Rev 3, Appendix B and Test Plan v1.0.

This appendix settles twelve contradictions and gaps found across the contract documents. Each item names the clause it settles or overrides, the resolution, and a one-li

ne rationale.

Precedence rule (binding): where Appendix C conflicts with the Spec, Phase 1, Appendix A, Appendix B or the Test Plan, Appendix C wins. Anything still ambiguous after Appendix C is raised before implementation (Spec §2), never resolved silently in code or tests.

Items marked [**Override**] change the literal text of an earlier document. Items marked [**Settle**] fill a gap or pick between readings without changing intent.

C1. Authoritative revisions [Settle]

Affects: Spec V2 footer ("Rev 3"), Phase 1 footer ("Rev 2"), Appendix B header ("companion to Spec V2 Rev 3").

- Spec V2 **Rev 4** and Phase 1 **Rev 3** are authoritative. The mismatched footers are typographical errors.
- Appendix B is binding against Rev 4. Where Appendix B text predates Rev 4 (see C6), Rev 4 wins.

Rationale: the headers and change logs are explicit; the footers were not updated.

C2. Linear and rotation rounding [Settle]

Affects: Spec §3 Quantization, Phase 1 §1 rounding mode, Test Plan §3 quantize properties.

- Rounding is **round half away from zero** on the millimetre value. This is the "same magnitude rule" the spec describes; JavaScript's bare `Math.round` (half toward $+\infty$) is not used for negatives.
- Reference implementation:

```
function quantizeLinear(m: number): number {
  const mm = Math.round(Math.abs(m) * 1000 + 1e-9);
  const q = Math.sign(m) * mm / 1000;
  return q === 0 ? 0 : q; // normalise -0 to 0
}
```

- The `1e-9` mm nudge makes decimal midpoints that are not exact in binary (for example 1.0005 m) round up, as a person would expect.
- `-0` never appears in committed state or serialized output.
- Rotation uses the same rule on tenths of a degree, before conversion to radians.
- Expected values: $0.0004 \rightarrow 0$, $0.0005 \rightarrow 0.001$, $1.0005 \rightarrow 1.001$, $-0.0005 \rightarrow -0.001$, $-0.0004 \rightarrow 0$.
- The quantize property tests use generators bounded to $|x| \leq 1000$ m.

Rationale: the spec's two descriptions disagree for negatives; the magnitude rule is the stated intent and is symmetric.

C3. Rotation range [Settle]

Affects: Spec §3 Quantization, Phase 1 `FurnitureInstance.rotation`.

- Committed rotation is normalised to `[0, 2 $\pi$ )` after quantization. 360.0° stores as 0.
- Comparisons of rotation in tests account for wraparound (359.95° and 0.05° are 0.1° apart).

Rationale: one canonical range keeps replay hashes and inverse checks stable.

C4. Touching vs. separated at EPSILON [Settle]

Affects: Spec Rule 17, §3 Floating-point policy, A10, Test Plan §3 "Touching $\neq$ collision".

- A gap or penetration whose absolute value is $\leq$ **EPSILON** (1e-6 m) is touching. Strictly greater than EPSILON is separated or overlapping.
- Property tests do not probe exactly EPSILON. They test at $0.5 \times$ EPSILON (must be touching) and $2 \times$ EPSILON (must be separated or overlapping).

Rationale: an inclusive boundary is the safer default for valid placements, and testing exactly at the boundary is inherently flaky in floating point.

C5. EditWall carries fixtures on both sides [Override]

Affects: Phase 1 §4 `EditWallCommand`, §6a, A3, Test Plan §2 `wall-edit-cascade` fixture.

- `from.fixtures` and `to.fixtures` are **mandatory**. Each holds the room's complete fixture list before and after the edit. `fixtureSnapshots?` is removed.
- Fixture offsets clamped by the interaction layer are carried in `to.fixtures`, so `apply` stays pure and undo restores fixtures exactly.
- Fixtures that cannot be clamped are carried unchanged and reported by re-validation as `fixture_out_of_wall` (derived, non-blocking).

```
interface EditWallCommand {
  type: 'EditWall';
  roomId: string;
  from: { vertices: Vertex[]; walls: WallSegment[]; fixtures: Fixture[] };
  to:   { vertices: Vertex[]; walls: WallSegment[]; fixtures: Fixture[] };
}
```

Rationale: the Rev 3 command had no place for post-edit fixture state, so the cascade fixture's "clamped per A3" expectation was unreachable at engine level.

C6. Vertex insertion and deletion by ID [Override]

Affects: Appendix B B1 ("Index remapping per A1"), Spec A1 vertex-ID rule.

- The B1 phrase "Index remapping per A1" is void. Wall topology uses stable vertex IDs only (Rev 4).
- **Insertion:** the new vertex gets a new ID. The split segment keeps its original ID for the half ending at the new vertex; the other half gets a new segment ID. Both halves inherit the original thickness.
- **Fixtures on a split wall:** a fixture whose centre falls on the new half has its `wallId` and `offsetAlongWall` updated, carried in `to.fixtures` (C5). A fixture straddling the new vertex is handled as an unclamped fixture (C5).
- **Deletion:** the two segments meeting at the deleted vertex merge into one segment that keeps the ID of the incoming segment (the one ending at the vertex) and its thickness. Fixtures on the outgoing segment are re-homed the same way.
- All new vertex and segment IDs are generated outside `apply` and carried in the command.

Rationale: Rev 4 replaced index topology; B1 was written against Rev 3 and was not updated.

C7. Collinear vertices are valid [Settle]

Affects: Spec §4 polygon validation, Phase 1 `PolygonErrorCode`.

- Three consecutive collinear vertices are **valid**. Vertex insertion (B1) creates them by design.
- `DEGENERATE_EDGE` means an edge shorter than 1 mm after quantization.
- `DUPLICATE_VERTEX` means two non-adjacent vertices at the same position (within EPSILON). Two adjacent vertices at the same position are `DEGENERATE_EDGE`.

Rationale: rejecting collinear vertices would make every B1 insertion invalid.

C8. Fixture position along the wall [Override]

Affects: Spec A1 and A4, Phase 1 `Fixture.offsetAlongWall`.

- `offsetAlongWall` is the distance from the wall's **start vertex** to the fixture's **centre**, measured along the interior face.
- **End margin:** each edge of the opening must stay at least **50 mm** from its wall end. Valid centre range is $[\text{width}/2 + 0.05, L - \text{width}/2 - 0.05]$, where L is the interior segment length.
- A fixture wider than $L - 0.1$ m cannot be placed on that segment.

Rationale: A4's literal wording ("half the fixture width or 50 mm, whichever is larger") allows a door edge flush into a corner, which leaves no room for a frame. **This changes the spec's text and is flagged for confirmation.**

C9. Added commands [Override]

Affects: Phase 1 §4 `Command` union; Spec A5, A9, A16, B4, B5, B8; Test Plan M5.

The Phase 1 command set has no way to express several required operations. The following are added; all follow the existing rules (immutable, quantized, IDs supplied by the caller, mandatory inverse data).

```
interface UpdateFurnitureCommand {
  type: 'UpdateFurniture';
  instanceId: string;
  from: Partial<FurnitureProps>; // REQUIRED: prior values of exactly the
  keys in `to`
  to: Partial<FurnitureProps>;
}
// FurnitureProps = elevation | height | locked | finishOverrides | metadata

interface UpdateFixtureCommand {
  type: 'UpdateFixture';
  fixtureId: string;
  from: Partial<Fixture>; // REQUIRED inverse data
```

```

    to:    Partial<Fixture>; // id and type may not change
  }

  interface DeleteFixtureCommand {
    type: 'DeleteFixture';
    fixtureId: string;
    snapshot: Fixture; // REQUIRED full pre-state
  }

  interface CreateRoomCommand {
    type: 'CreateRoom';
    room: Room; // full room, pre-generated IDs, validated polygon
    // inverse: DeleteRoom with snapshot = room
  }

  interface DeleteRoomCommand {
    type: 'DeleteRoom';
    roomId: string;
    snapshot: Room; // REQUIRED full pre-state incl. walls, fixtures, furniture
  }

```

Mapping of operations to commands:

Operation	Command
Edit metadata, finish, elevation, height	UpdateFurniture
Lock / unlock	UpdateFurniture (locked)
Multi-selection inspector edit (B4)	Composite of UpdateFurniture / ResizeFurniture / RotateFurniture
Duplicate (single)	PlaceFurniture
Duplicate (group)	Composite of PlaceFurniture
Fix position (B5)	MoveFurniture (see C10)
Start from a rectangle (B8)	CreateRoom
Move fixture along wall, change swing	UpdateFixture

- UpdateFurniture may target a locked instance only to change locked itself. All other edits on a locked instance are rejected in the pipeline (A16).
- width, length and position stay in ResizeFurniture / MoveFurniture; UpdateFurniture may not carry them.
- A from / to key mismatch is STRUCTURALLY_CORRUPT_COMMAND.

Rationale: each operation above is required by a binding clause but had no command, so an implementer would otherwise invent one.

C10. Fix position [Settle]

Affects: Spec A3 no-deadlock rule, Appendix B B5, Phase 1 §6 escape rule.

- "Nearest valid position" is computed by **translation only**. Rotation and size are kept.
- "Nearest" is the smallest Euclidean distance of the object's centre from its current position, among quantized positions with no hard violation.
- Search is deterministic: candidate positions on a 1 mm grid, expanding outward in rings, ties broken by smallest angle measured counter-clockwise from +X. Implementations may use a faster method if it returns the same result.
- The search is bounded to the room's bounding box. If no valid position exists (for example, the room is now smaller than the object), the action shows "No valid position — resize or move manually" and **no command is emitted**.
- A successful Fix position is a single `MoveFurniture` and one history entry.

Rationale: translation-only keeps the result predictable for the designer, and the no-result case was previously undefined.

C11. Commit pipeline module [Settle]

Affects: Phase 1 §5 module list, §6, Test Plan §1 structure.

- A new pure module `pipeline.ts` owns the canonical commit pipeline: resolve snap → quantize → lock check → validate → escape rule (for already-invalid objects) → emit command.
- It returns either a command or a structured rejection (`{ rejected: true, violations }`); it never mutates state.
- `EditWall` follows the §6a exception inside the same module.
- Tests live in `tests/unit/pipeline.test.ts`.

Rationale: the pipeline holds the most important rules but had no home in the module list.

C12. Test expectations [Settle]

Affects: Test Plan §2 fixture rules, §3 inverse and replay properties.

- **Expected results are derived by hand**, with the working shown in a comment or sidecar note. They are never generated by running the engine under test. A scenario whose expectation was produced by the engine is invalid.

- **Violation matching:** compare `type` , `severity` and each ID list sorted. Order of `violations[]` and `message` text are not compared.
- **One equality helper:** all project comparisons use a shared `expectProjectEqual(a, b)` . Linear values compare exactly; rotation compares within 1e-9 rad with wraparound (C3).
- **Replay hash:** canonical JSON (keys sorted, rotation rounded to 1e-9 rad before hashing), then SHA-256.
- **Naming:** test data directory is `tests/scenarios/` (not `fixtures/`) to avoid confusion with door and window fixtures.

Rationale: engine-generated expectations make every test pass by definition, and the Test Plan did not define how results are compared.

Change log

Item	Type	Needs confirmation
C1 Authoritative revisions	Settle	No
C2 Rounding	Settle	No
C3 Rotation range	Settle	No
C4 EPSILON boundary	Settle	No
C5 EditWall fixtures	Override	No
C6 Vertex insert/delete by ID	Override	No
C7 Collinear vertices	Settle	No
C8 Fixture offset and end margin	Override	Yes — changes A4 wording
C9 Added commands	Override	No
C10 Fix position	Settle	Yes — defines "nearest"
C11 Pipeline module	Settle	No
C12 Test expectations	Settle	No

At the next Spec revision (Rev 5), these resolutions are folded into the main documents and this appendix is retired.

End of Appendix C (Binding for P0)